

Baseball Card Tracker

[![Java](https://img.shields.io/badge/Java-17+-blue.svg)](https://www.oracle.com/java/)

[![Spring Boot](https://img.shields.io/badge/Spring%20Boot-3.x-green.svg)](https://spring.io/projects/spring-boot)

[![PostgreSQL](https://img.shields.io/badge/PostgreSQL-14+-blue.svg)](https://www.postgresql.org/)

> **CSCE 548 - Secure Software Development Project**

> An n-tier web application for managing baseball card collections built with Java, Spring Boot, and PostgreSQL.

📄 Table of Contents

- [Overview](#overview)
- [Features](#features)
- [Architecture](#architecture)
- [Technologies Used](#technologies-used)
- [Prerequisites](#prerequisites)
- [Installation](#installation)
- [Configuration](#configuration)
- [Running the Application](#running-the-application)
- [API Endpoints](#api-endpoints)
- [Testing](#testing)
- [Project Structure](#project-structure)
- [Documentation](#documentation)

- [License](#license)

🌟 Overview

The **Baseball Card Tracker** is a full-stack web application that allows users to manage their baseball card collections. The application follows an **n-tier architecture** with separate presentation, service, business, and data layers. Users can create, read, update, and delete baseball cards, manage card styles, and search their collection.

This project was developed as part of the CSCE 548 Secure Software Development course, demonstrating the complete software development lifecycle from requirements to deployment.

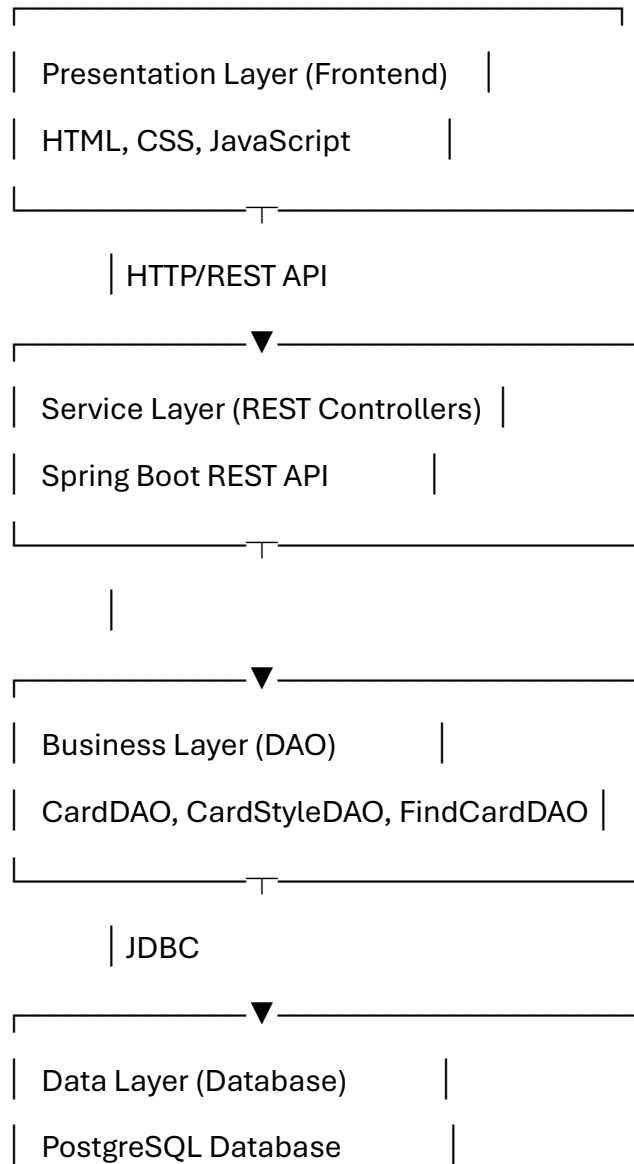
✨ Features

- **Create Baseball Cards**: Add new cards with player information (name, team, position, stats)
- **View All Cards**: Browse your complete collection in an interactive grid layout
- **Update Cards**: Modify existing card details
- **Delete Cards**: Remove cards from your collection
- **Search Functionality**: Find specific cards by player name or team
- **Card Styles Management**: Track card manufacturers, years, and editions
- **Responsive UI**: Modern, user-friendly web interface
- **RESTful API**: Well-documented REST endpoints for all operations

🏗️ Architecture

This application uses a **4-tier architecture**:

...



|

...

Technologies Used

Backend

- **Java 17+** - Core programming language
- **Spring Boot 3.x** - Application framework
- **Maven** - Dependency management and build tool
- **PostgreSQL** - Relational database
- **JDBC** - Database connectivity

Frontend

- **HTML5** - Page structure
- **CSS3** - Styling and layout
- **JavaScript (ES6+)** - Client-side logic
- **Fetch API** - HTTP requests

Development Tools

- **IntelliJ IDEA / VS Code** - IDE
- **pgAdmin 4** - Database administration
- **Git** - Version control

📦 Prerequisites

Before you begin, ensure you have the following installed:

1. **Java Development Kit (JDK) 17 or higher**

- Download: [Oracle JDK](https://www.oracle.com/java/technologies/downloads/) or [OpenJDK](https://openjdk.org/)
- Verify: `` java -version ``

2. **Apache Maven 3.6+**

- Download: [Maven](https://maven.apache.org/download.cgi)
- Verify: `` mvn -version ``

3. **PostgreSQL 14+**

- Download: [PostgreSQL](https://www.postgresql.org/download/)
- Verify: `` psql --version ``

4. **Git**

- Download: [Git](https://git-scm.com/downloads)
- Verify: `` git --version ``

5. **Web Browser** (Chrome, Firefox, Edge, or Safari)

🚀 Installation

Step 1: Clone the Repository

```
` `` bash
git clone https://github.com/colinelmor/SecureSoftware.git
cd SecureSoftware/baseball-card-tracker
` ``
```

Step 2: Set Up the Database

1. ****Start PostgreSQL**** (if not already running)
2. ****Create the database**** (or use existing `postgres` database):

```
` `` bash
psql -U postgres
CREATE DATABASE baseball_cards;
\q
` ``
```

3. ****Run the database setup scripts****:

```
` `` bash
# Navigate to database folder
cd database

# Create tables
```

```
psql -U postgres -d postgres -f create_tables.sql
```

```
# Insert test data (optional)
```

```
psql -U postgres -d postgres -f insert_test_data.sql
```

```
```
```

### ### Step 3: Configure Database Connection

Edit `src/main/resources/application.properties` with your database credentials:

```
```properties
```

```
database.url=jdbc:postgresql://localhost:5432/postgres
```

```
database.username=your_username
```

```
database.password=your_password
```

```
server.port=8080
```

```
```
```

**\*\*Important\*\*:** Update `your\_username` and `your\_password` with your PostgreSQL credentials.

### ### Step 4: Build the Project

```
```bash
```

```
# From the baseball-card-tracker directory
```

```
mvn clean install
```

```
```
```

This command will:

- Download all dependencies
- Compile the Java source code
- Run unit tests
- Package the application as a JAR file

---

## ## Configuration

### ### Database Configuration

The application connects to PostgreSQL using settings in `application.properties`:

| Property            | Description         | Default                                     |
|---------------------|---------------------|---------------------------------------------|
| -----               | -----               | -----                                       |
| `database.url`      | JDBC connection URL | `jdbc:postgresql://localhost:5432/postgres` |
| `database.username` | Database username   | `postgres`                                  |
| `database.password` | Database password   | _(update this)_                             |
| `server.port`       | Application port    | `8080`                                      |

### ### Frontend Configuration

The frontend connects to the backend API via:



```
File: `src/main/resources/static/app.js`
`` `javascript
const API_URL = 'http://localhost:8080/api/cards';
`` `
```

Update this URL if deploying to a different host or port.

---

##  Running the Application

### Option 1: Using Maven (Recommended)

```
`` `bash
From baseball-card-tracker directory
mvn spring-boot:run
`` `
```

### Option 2: Using the JAR file

```
`` `bash
Build first
mvn clean package

Run the JAR
java -jar target/baseball-card-tracker-1.0.0.jar
```

```

Option 3: Using IDE

1. Open the project in IntelliJ IDEA or Eclipse
2. Navigate to `src/main/java/com/baseballcards/Application.java`
3. Right-click and select ****Run 'Application'****

🌐 Accessing the Application

Once the application starts, you'll see:

```

=====

Baseball Card Service Started!

=====

API Base URL: <http://localhost:8080/api/cards>

Health Check: <http://localhost:8080/api/cards/health>

=====

```

****Open your browser and navigate to:****

- ****Frontend****: <http://localhost:8080>

- ****API Health Check****:

http://localhost:8080/api/cards/health

🚧 API Endpoints

Card Management

Method	Endpoint	Description
GET	/api/cards	Get all baseball cards
GET	/api/cards/{firstName}/{lastName}	Get card by player name
POST	/api/cards	Create a new card
PUT	/api/cards	Update an existing card
DELETE	/api/cards/{firstName}/{lastName}	Delete a card

Card Styles

Method	Endpoint	Description
GET	/api/styles	Get all card styles
POST	/api/styles	Create a new style
PUT	/api/styles	Update a style

Search

Method	Endpoint	Description
--------	----------	-------------

Method	Endpoint	Description
--------	----------	-------------

POST	/api/cards/search	Search cards by criteria
------	-------------------	--------------------------

Example API Request

Create a new card:

```
bash
```

```
curl -X POST http://localhost:8080/api/cards \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "firstName": "Mike",
```

```
  "lastName": "Trout",
```

```
  "height": 74.0,
```

```
  "weight": 235.0,
```

```
  "position": "OF",
```

```
  "team": "Los Angeles Angels",
```

```
  "battingAverage": 0.305
```

```
}'
```

```
```\
```

---

##  Testing

### ### Run Unit Tests

```
```bash
```

```
mvn test
```

```
```
```

### ### Manual Testing

See the **\*\*System Test Documentation\*\*** ( `System\_Test.pdf` ) in the repository root for detailed test cases and screenshots.

---

## ## 📁 Project Structure

```
...
```

```
baseball-card-tracker/
```

```
├── database/ # Database scripts
| ├── create_tables.sql # Table creation script
| ├── insert_test_data.sql # Sample data
| └── schema.sql # Schema definition
├── src/
| ├── main/
| | ├── java/com/baseballcards/
| | | ├── Application.java # Spring Boot entry point
| | | └── controllers/ # REST controllers
```

```

| | | └─ dao/ # Data Access Objects
| | | └─ models/ # Data models
| | └─ resources/
| | └─ application.properties # Configuration
| | └─ static/ # Frontend files
| | └─ index.html # Main HTML page
| | └─ app.js # JavaScript logic
| | └─ styles.css # Styling
| └─ test/ # Unit tests
└─ pom.xml # Maven configuration
└─ README.md # This file
...

```

---

## ## 📄 Documentation

Additional documentation can be found in the repository:

- **\*\*[Deployment Document](./Deployment\_Document.pdf)\*\*** - Complete deployment guide with step-by-step instructions
- **\*\*[System Test Documentation](./System\_Test.pdf)\*\*** - Full system test cases with screenshots
- **\*\*[Project Specification](./Project4.pdf)\*\*** - Original project requirements

---

## ## 🎓 Course Information

- **Course**: CSCE 548 - Secure Software Development
- **Project**: Baseball Card Tracker (Projects 1-4)
- **Semester**: Spring 2026
- **Institution**: University of South Carolina

---

## ## 👤 Author

**Colin Elmore**

- GitHub: [@colinelmore](https://github.com/colinelmore)
- Repository: [SecureSoftware](https://github.com/colinelmore/SecureSoftware)

---

## ## 📄 License

This project is developed for educational purposes as part of CSCE 548.

---

## ## 🆘 Troubleshooting

### ### Common Issues

#### \*\*1. Database Connection Failed\*\*

- Verify PostgreSQL is running: ``pg_isready``
- Check credentials in ``application.properties``
- Ensure database exists: ``psql -U postgres -l``

#### \*\*2. Port 8080 Already in Use\*\*

- Change port in ``application.properties``: ``server.port=8081``
- Or kill the process using port 8080

#### \*\*3. Maven Build Fails\*\*

- Clean and rebuild: ``mvn clean install -U``
- Check Java version: ``java -version`` (must be 17+)

#### \*\*4. Frontend Can't Connect to API\*\*

- Verify backend is running
- Check browser console for CORS errors
- Confirm API\_URL in ``app.js`` matches your server

---

### ## Verification

To verify successful installation:



1. ✅ Backend starts without errors
2. ✅ Navigate to `http://localhost:8080` - see the Baseball Card Tracker interface
3. ✅ Click "All Cards" - see cards loaded from database
4. ✅ Create a new card - verify it appears in the database
5. ✅ API health check returns 200 OK: `http://localhost:8080/api/cards/health`

---

## ## 🙏 Acknowledgments

- Project developed using AI assistance (ChatGPT, GitHub Copilot)
- Spring Boot framework and documentation
- PostgreSQL community

---

**\*\*Last Updated\*\***: March 2026